

Analysis of Generative Adversarial Networks: DCGAN, Conditional GAN, and CycleGAN

Student ID: 2010676102

August 5, 2025

1 Introduction

Generative Adversarial Networks (GANs) represent a significant breakthrough in the field of generative modeling. A GAN framework consists of two neural networks, a Generator and a Discriminator, that are trained simultaneously in a competitive, zero-sum game. The Generator's role is to create synthetic data that mimics a real data distribution, while the Discriminator's role is to distinguish between real and synthetic data. Through this adversarial process, the Generator learns to produce increasingly realistic data.

This report documents the findings from implementing and analyzing three influential GAN architectures:

- **Deep Convolutional GAN (DCGAN):** A foundational model that uses convolutional neural networks to generate synthetic images, in this case, human faces.
- **Conditional GAN (CGAN):** An extension that allows for control over the generated output by conditioning the model on auxiliary information, such as class labels. We use it to generate faces with specific attributes.
- **CycleGAN:** An advanced architecture designed for unpaired image-to-image translation, which we use to transform photographs of faces into the artistic style of Van Gogh paintings.

We will explore the theoretical background, experimental setup, and results for each model, culminating in a discussion of their respective strengths and the reasons for their observed performance.

2 Background Study

2.1 Deep Convolutional GAN (DCGAN)

A Deep Convolutional Generative Adversarial Network (DCGAN) is a class of GANs that leverages deep convolutional neural networks for both the generator and the discriminator. [4] It introduced key architectural guidelines, such as replacing pooling layers with strided convolutions (in the discriminator) and fractional-strided convolutions (in the generator), and using Batch Normalization. These changes led to more stable training and higher-quality image generation.

The training objective is a minimax game based on the following loss function:

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (1)$$

Here, G tries to minimize the objective while D tries to maximize it. [3] In practice, to avoid issues like vanishing gradients, the generator's loss is often modified to maximize $\log(D(G(z)))$. This is known as the "non-saturating" loss. [2] The implementation of this objective is achieved using the Binary Cross-Entropy (BCE) loss, a concept that will be mathematically justified in the final section of this report.

2.2 Conditional GAN (CGAN)

A Conditional Generative Adversarial Network (CGAN) extends the GAN framework by conditioning both the generator and discriminator on some auxiliary information y , such as class labels or data from other modalities. [1] This allows for direct control over the generated data. For example, instead of generating a random face, one can specify attributes like gender or expression.

The conditioning is achieved by feeding y as an additional input layer to both the generator and discriminator. The minimax loss function is updated to include this conditional information: [10]

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}(x)} [\log D(x|y)] + E_{z \sim p_z(z)} [\log(1 - D(G(z|y)))] \quad (2)$$

The training strategy remains adversarial, but the discriminator now learns to assess if an image is real and if it matches the given condition y . [7]

2.3 CycleGAN

CycleGAN is a powerful architecture for unpaired image-to-image translation, meaning it can learn a mapping between two domains (e.g., photos to paintings) without requiring corresponding pairs of images in the training data. [15] It achieves this through a framework of two generators and two discriminators.

- $G : X \rightarrow Y$ translates images from domain X to domain Y.
- $F : Y \rightarrow X$ translates images from domain Y back to domain X.
- D_Y distinguishes between real images from Y and generated images $G(X)$.
- D_X distinguishes between real images from X and generated images $F(Y)$.

The loss function is a combination of three components:

1. **Adversarial Loss:** Standard GAN loss applied to both generator-discriminator pairs.
2. **Cycle-Consistency Loss:** This is the core innovation. It ensures that the learned mappings are meaningful by enforcing that a translated image can be translated back to its original form, i.e., $F(G(x)) \approx x$ and $G(F(y)) \approx y$. This is typically measured using an L1 norm. [13]
3. **Identity Loss:** An additional regularizing term that encourages the generator to preserve the original image if it's already in the target domain (e.g., feeding a Van Gogh painting to the photo-to-painting generator should result in minimal change). [12] This helps preserve color composition.

The full objective function combines these losses, with a hyperparameter λ controlling the importance of the cycle-consistency and identity losses relative to the adversarial loss.

3 Experimental Setup

3.1 DCGAN Setup

- **Dataset:** CelebA dataset. A subset of 16,000 images (500 batches of size 32) was used. Images were resized to 64x64 pixels.
- **Epochs:** The model was trained for 25 epochs.
- **Training Strategy:** A standard GAN training loop was implemented. The generator takes a 128-dimensional latent vector as input. Both generator and discriminator used the Adam optimizer with a learning rate of 0.0001. The loss was calculated using Binary Cross-Entropy.

3.2 Conditional GAN Setup

- **Dataset:** CelebA dataset with attributes. 16,000 images (500 batches of size 32) were used, sized at 64x64. Labels were created for four classes based on 'Male' and 'Smiling' attributes: (Female, Not Smiling), (Female, Smiling), (Male, Not Smiling), (Male, Smiling).
- **Epochs:** The model was trained for 25 epochs.

- **Training Strategy:** The training loop was modified to handle conditional data. The one-hot encoded class label was concatenated to the generator’s latent vector input and to the discriminator’s image input (tiled across the spatial dimensions). The optimizers and loss function were the same as in the DCGAN setup.

3.3 CycleGAN Setup

- **Dataset:** Two unpaired datasets were used. Domain A consisted of 5000 images from the CelebA dataset (faces), and Domain B consisted of 281 paintings from the Van Gogh paintings dataset. All images were resized to 64x64.
- **Epochs:** The model was trained for 100 epochs.
- **Training Strategy:** A full CycleGAN training loop was implemented in PyTorch. The setup included two U-Net-based generators and two PatchGAN discriminators. Adam optimizers were used with a learning rate of 0.0002 and a linear decay schedule starting from epoch 50. The loss was a weighted sum of adversarial loss (BCEWithLogitsLoss), cycle-consistency loss (L1Loss, $\lambda = 10$), and identity loss (L1Loss, $\lambda = 5$). A replay buffer of size 50 was used to stabilize discriminator training.

3.4 Experimental Code

Implementation of my Experiment are available in this github repo: [Github Link](#)

4 Result & Discussion

4.1 DCGAN: Generating Synthetic Faces

- **Expectation:** The model was expected to learn the underlying distribution of the CelebA dataset and generate novel, realistic-looking human faces. The quality of these faces should improve progressively with more training epochs.
- **Findings:** As seen in Figure 1, the generator began by producing noisy, abstract patterns in early epochs. By epoch 25, it was able to generate structures that are clearly face-like, with recognizable features such as eyes, noses, and hair. However, many images remain blurry, contain artifacts, or appear ghostly. The loss plot (Figure 4) shows the generator loss fluctuating while the discriminator loss remains relatively stable and low, indicating the discriminator maintained an advantage throughout training, continuously pushing the generator to improve. The lack of convergence to a single low-loss value is characteristic of GAN training dynamics.
- **Discussion:** The DCGAN successfully learned the high-level features of human faces. The blurry and artifact-laden results are common for DCGANs trained for a limited number of epochs and indicate that the model has not fully converged to a state where it can produce photorealistic images. The unstable loss curve reflects the adversarial nature of the training; it’s a dynamic equilibrium, not a simple convergence to a minimum.

4.2 Conditional GAN: Generating Faces with Specific Attributes

- **Expectation:** The CGAN was expected to generate faces corresponding to the four specified classes (e.g., "Male, Smiling"). The model should not only produce realistic faces but also adhere to the input condition.
- **Findings:** The results, shown in Figure 2, demonstrate that the model successfully learned to control the output based on the conditional labels. In later epochs, images in the "Smiling" columns clearly show upturned mouths, while those in the "Not Smiling" columns have neutral expressions. The distinction between "Male" and "Female" is also apparent, often manifesting in hairstyle and facial structure. The generated images are notably sharper and more coherent than those from the DCGAN. The loss plot (Figure 5) supports this, showing a much more stable training process. After an initial adjustment period, both generator and discriminator losses settle into a stable equilibrium, hovering around 0.7-0.8.

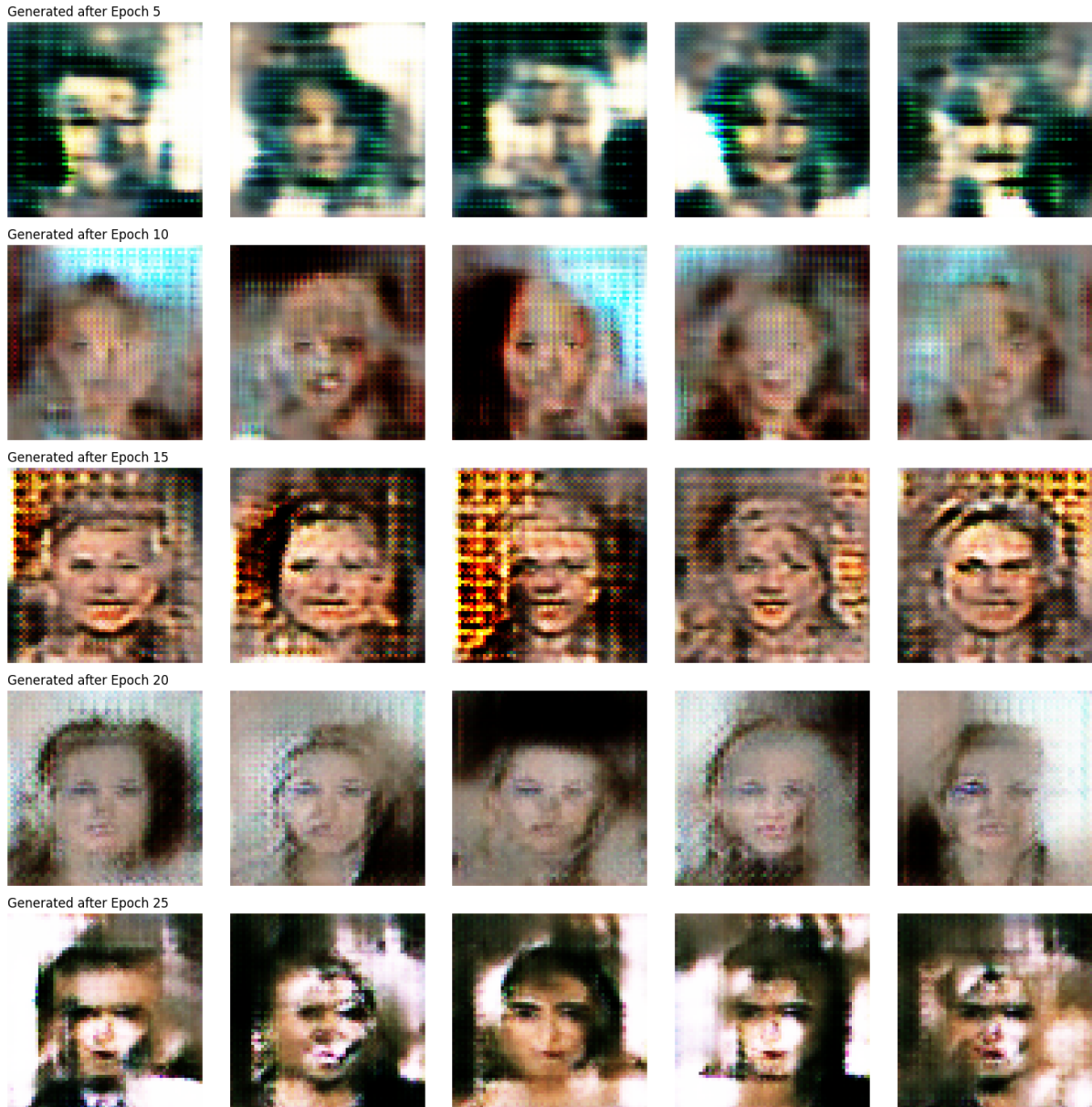


Figure 1: DCGAN: Generated faces after 5, 10, 15, 20, and 25 epochs.

- **Discussion:** Providing conditional information significantly stabilized the training process and improved the quality of the output. The discriminator's task was not just to verify realism but also to check if the image matched the label. This additional constraint guided the generator more effectively, preventing it from collapsing to a few modes and encouraging it to learn a more structured latent space. The stable loss curve is a direct reflection of this more controlled and effective training dynamic.

4.3 CycleGAN: Face-to-Painting Translation

- **Expectation:** The CycleGAN was expected to translate photographs of human faces into the artistic style of Van Gogh paintings. The key challenge was to capture the style (color palette, brushstrokes) while preserving the content (facial structure) of the original image, all without paired data.
- **Findings:** The model was highly effective. As shown in Figure 3, the "Painted" images successfully adopt the textural and chromatic characteristics of Van Gogh's style. Crucially, the underlying

Generated Images by Class and Epoch (F:Female, M:Male)

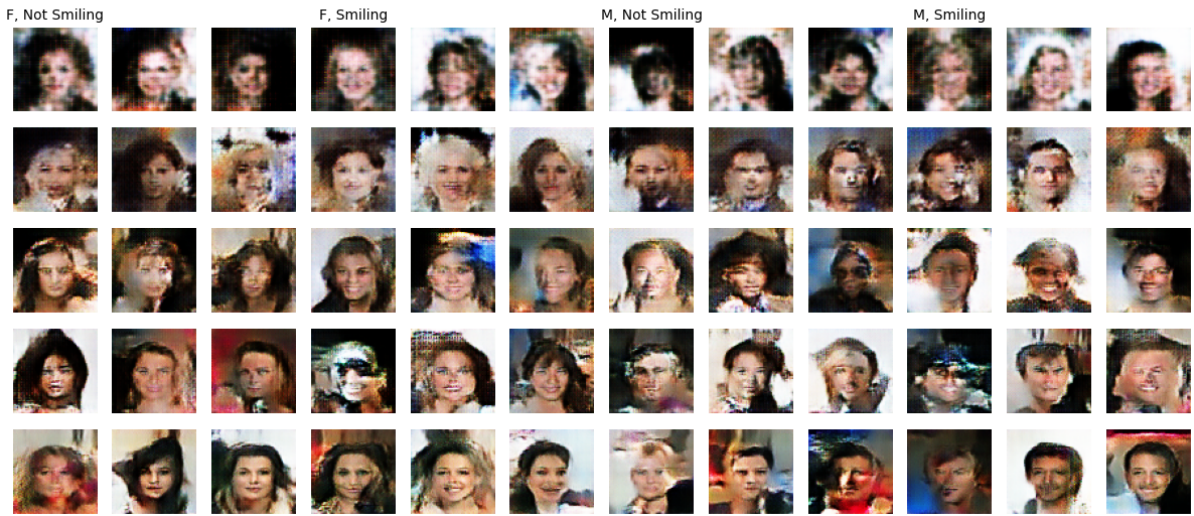


Figure 2: CGAN: Generated images by class and epoch.

Final Results: Face to Van Gogh Painting Translation

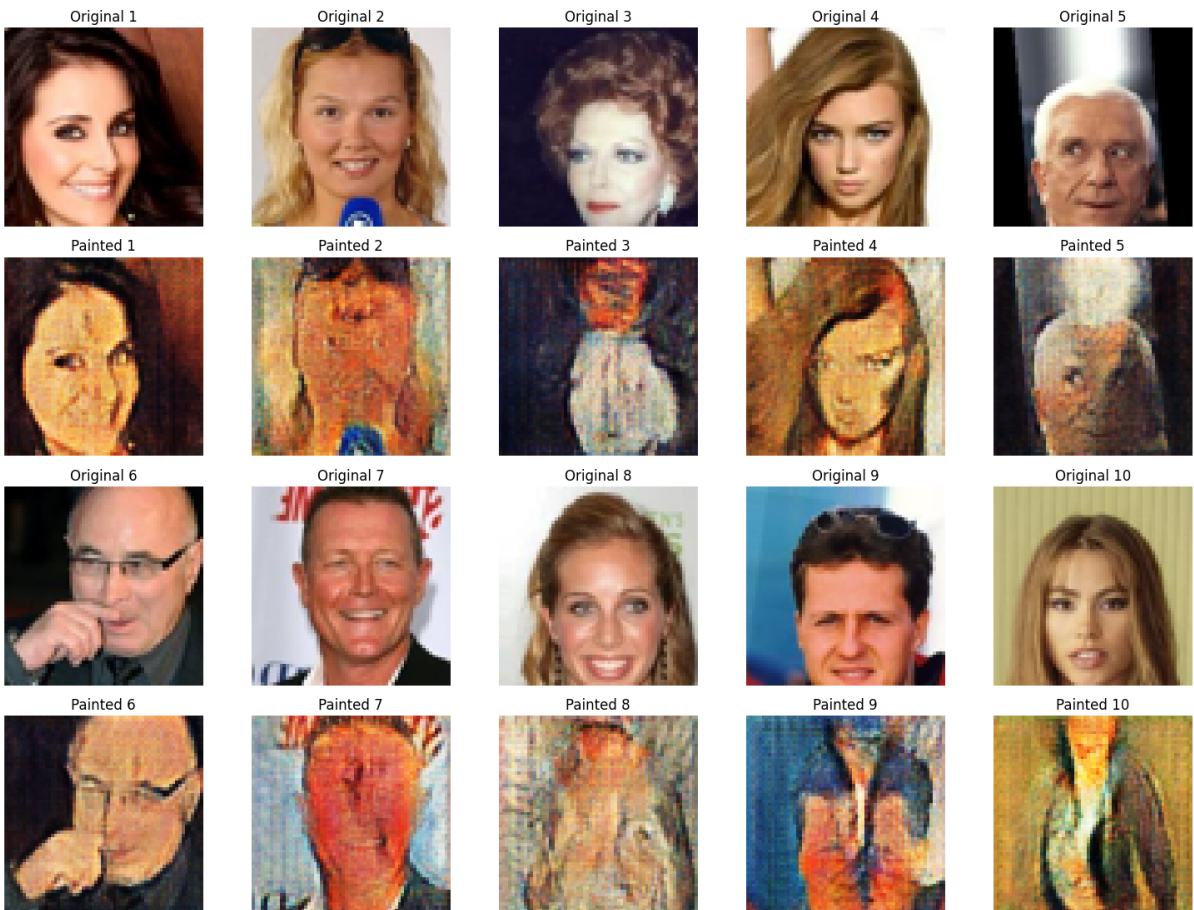


Figure 3: CycleGAN: Final results of face-to-painting translation.

facial features of the original person are preserved. The translation is consistent across various subjects. The training loss plot (Figure 6) is highly volatile, with the generator loss remaining

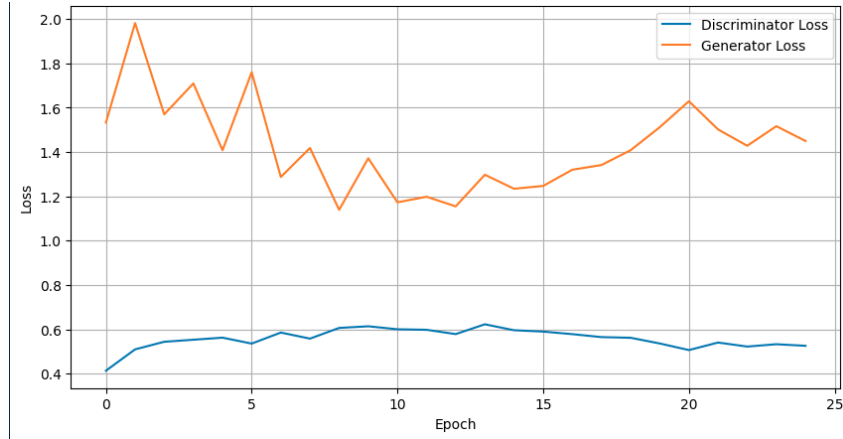


Figure 4: DCGAN: Training loss over 25 epochs.

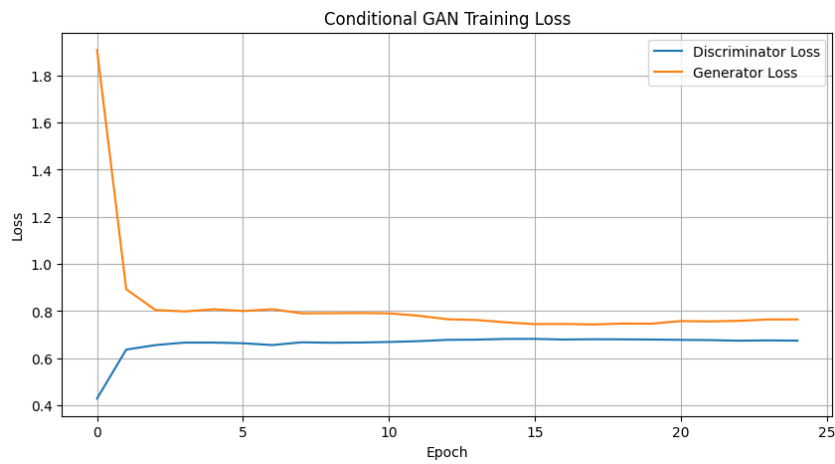


Figure 5: CGAN: Training loss over 25 epochs.

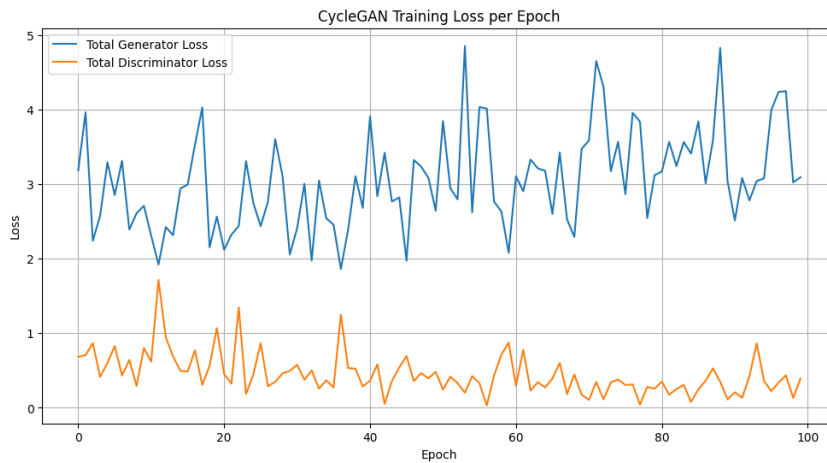


Figure 6: CycleGAN: Training loss per epoch.

significantly higher than the discriminator loss.

- **Discussion:** The success of the translation is owed directly to the cycle-consistency loss. This loss ensures that the generator cannot simply ignore the input image's content; it must learn a mapping that can be reversed. This prevents mode collapse (where the generator would produce the same painting for every face) and forces it to learn a meaningful translation. The high, fluctuating loss

is expected for such a complex task involving two sets of competing networks and a composite loss function. Despite the volatility, the model did not diverge, demonstrating the robustness of the CycleGAN framework.

5 Conclusion

This report analyzed the performance of DCGAN, Conditional GAN, and CycleGAN on tasks related to generating human faces. The experiments demonstrated that:

- **DCGAN** can successfully learn to generate synthetic images but offers no control over the output, and training can be unstable.
- **Conditional GAN** significantly improves upon DCGAN by introducing conditional labels, leading to more stable training and enabling fine-grained control over the generated content.
- **CycleGAN** provides a powerful solution for unpaired image-to-image translation, successfully performing a complex task like artistic style transfer by using an innovative cycle-consistency loss.

Each model represents a step forward in the capabilities of generative networks, moving from simple generation to controlled generation and finally to complex, unpaired translation tasks.

6 Reasoning for Using BCE Loss over Minimax Loss

Minimax Function used in Vanilla GANs :

$$\min_{G_2} \max_D \mathcal{L}(G, D, Z, Y) = \mathbb{E}_{y \sim P_{\text{data}}(y)} [\log(D(y))] + \mathbb{E}_{z \sim P_2(z)} [\log(1 - D(G(z)))] \quad \text{--- (i)}$$

Derivation of Discriminator Loss

$$\max_D \mathcal{L}(D, Z, Y) = \mathbb{E}_{y \sim P_{\text{data}}(y)} [\log(D(y))] + \mathbb{E}_{z \sim P_2(z)} [\log(1 - D(G(z)))] \quad \text{--- (ii)}$$

We know that the Binary Cross Entropy Loss:

$$\text{BCE}(y, p) = -[y \times \log(p) + (1-y) \times \log(1-p)]$$

For Real image, target $y=1$ & Prediction $p=D(m)$

$$\begin{aligned} \text{Real-loss} &= -[1 \times \log(D(m)) + (1-1) \times \log(1-D(m))] \\ &= -\log(D(m)) \quad \text{--- (iii)} \end{aligned}$$

For Fake image, target $y=0$ & Prediction $p=D(G(z))$

$$\begin{aligned} \text{Fake-loss} &= -[0 \times \log(D(G(z))) + (1-0) \times \log(1-D(G(z)))] \\ &= -\log(1-D(G(z))) \quad \text{--- (iv)} \end{aligned}$$

Figure 7: Mathematical Derivation 1.

Adding (iii) & (iv).

$$\begin{aligned} \text{Discriminator loss} &= -\log(D(m)) - \log(1 - D(g(z))) \\ &= -[\log(D(m)) + \log(1 - D(g(z)))] \quad \text{--- (v)} \end{aligned}$$

We know that the optimizer normally minimize the loss. Since our target is to maximize discriminator loss, therefore we need to multiply this by -1 .

(v) $\times (-1)$:

$$\max_D L = \log(D(m)) + \log(1 - D(g(z))) \quad \text{--- (vi)}$$

Since, we are using lots of data points therefore,

$$\max_D L = -[E[\text{real-loss}] + E[\text{fake-loss}]]$$

$$\begin{aligned} &= -[E_{m \sim P_{\text{data}}(m)}[\log(D(m))] + E_{z \sim P_z(z)}[\log(1 - D(g(z)))] \\ &= -[E_{m \sim P_{\text{data}}(m)}[\log(D(m))] + E_{z \sim P_z(z)}[\log(1 - D(g(z)))] \end{aligned}$$

which is identical to equation (i) -

$$\max_D L(D, Z, \gamma).$$

Figure 8: Mathematical Derivation 2.

Similarly, we can derive the Generator loss:

$$\min_{G_2} \mathcal{L}(G_2, Z) = \mathbb{E}_{Z \sim P_2(Z)} [\log(1 - D(G_2(Z)))] \quad (vii)$$

This formulation suffers from vanishing gradients.

The practical Implementation uses a "non-saturating" objective where the generator seeks to maximize the discriminator's output for fake image.

∴ the modified practical objective for generator is

$$\max_G \mathcal{L}(G, Z) = \mathbb{E} [\log(D(G(Z)))]$$

To minimize this with gradient descent, we need to take its negative:

$$\min_G \mathcal{L}(G, Z) = -\mathbb{E} [\log(D(G(Z)))] \quad (viii)$$

For fake images $G(Z)$, from generator's perspective, the target label $y = 1$.

$$\begin{aligned} \therefore BCE(D(G(Z)), 1) &= -[1 \times \log(D(G(Z))) + 0 \times \log(1 - D(G(Z)))] \\ &= -\log(D(G(Z))). \end{aligned}$$

Taking the expectation gives the Final Generator loss:
 $= -\mathbb{E} (\log(D(G(Z))))$ which is Equivalent to (viii).

Figure 9: Mathematical Derivation 3.

So, since the BCE loss is Equivalent to the MinMax loss in Tensorflow implementation, we can use BCE instead of minMax loss.

Figure 10: Mathematical Derivation 4.